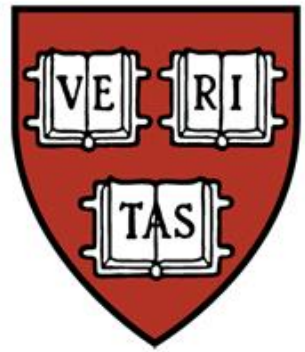


The Institute
for Quantitative Social Science
at Harvard University



Improving Metadata Workflow in a Data Repository With AI-Generated Metadata Recommendations

Bob Treacy – Principal Software Architect and Data Engineer

The Institute for Quantitative Social Science at Harvard University

Agenda

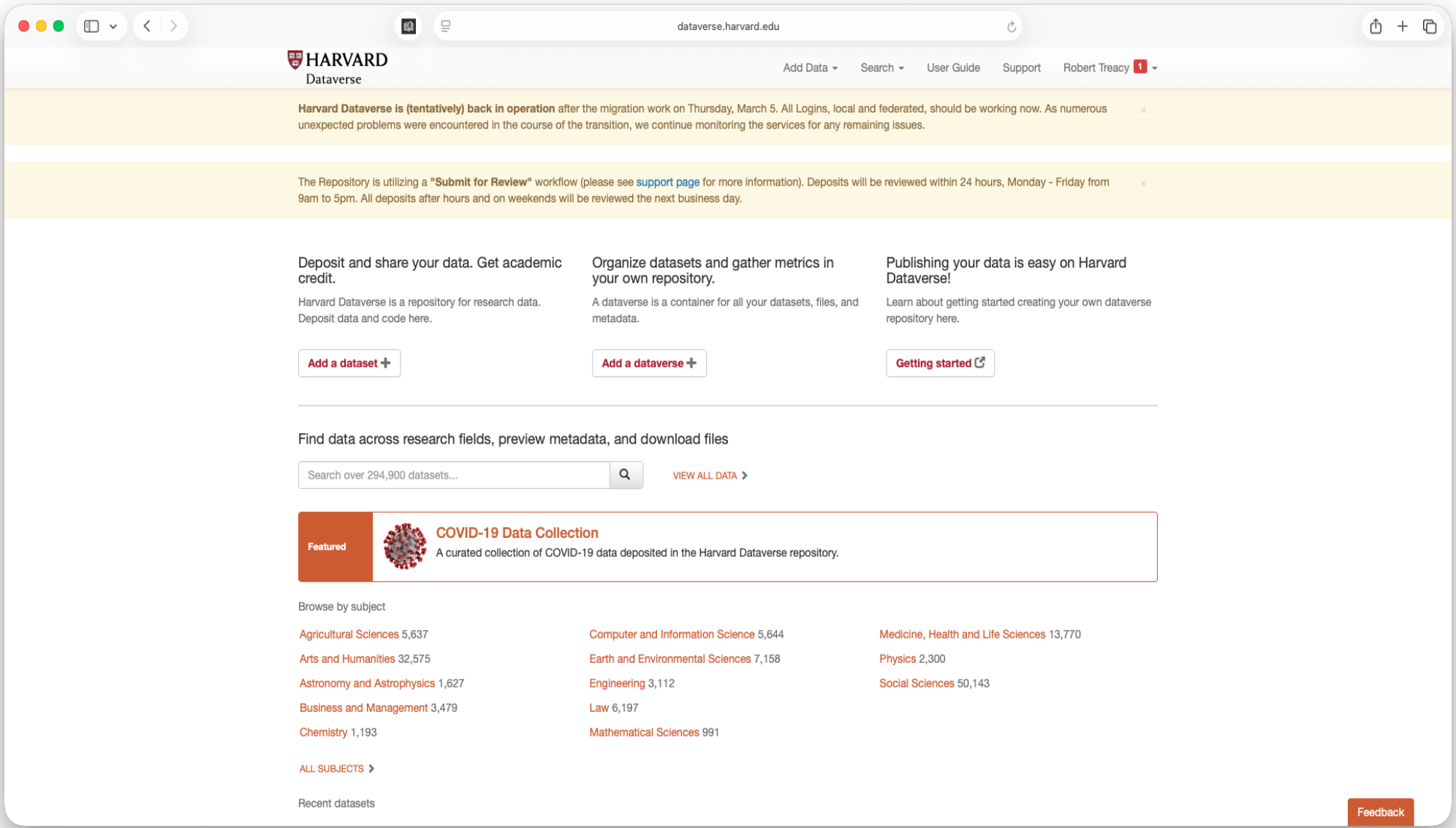
- Overview of Dataverse
- FAIR
- Retrieval Augmented Generation (RAG)
- Knowledge Graph using Neo4J
- Embeddings
- Similarity Metrics
- Demo
- Q&A

Dataverse

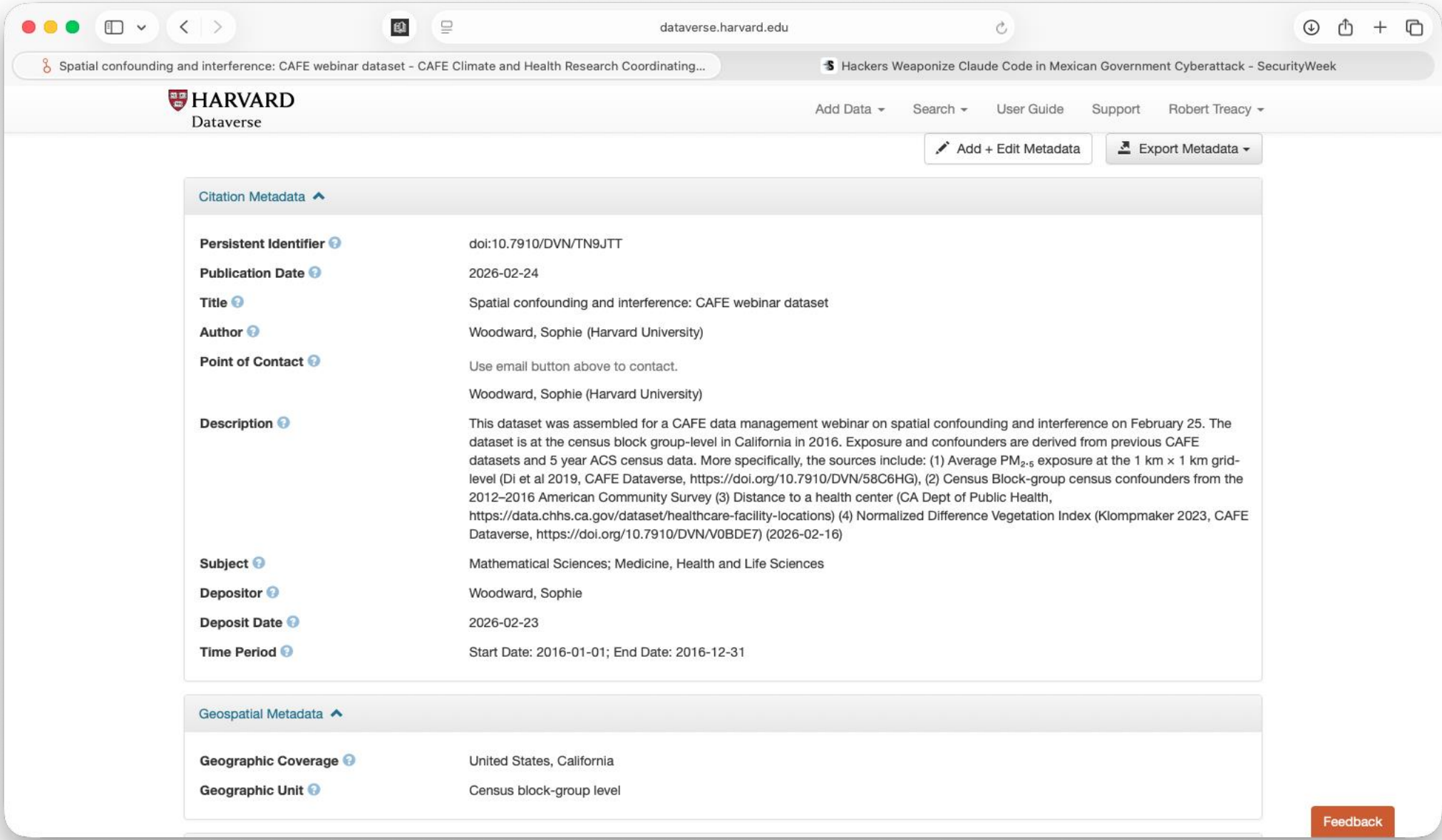
- open source web application to share, preserve, cite, explore, and analyze research data
- facilitates making data available to others, and allows you to replicate others' work more easily
- Researchers, journals, data authors, publishers, data distributors, and affiliated institutions all receive academic credit and web visibility

- https://dataverse.org/about

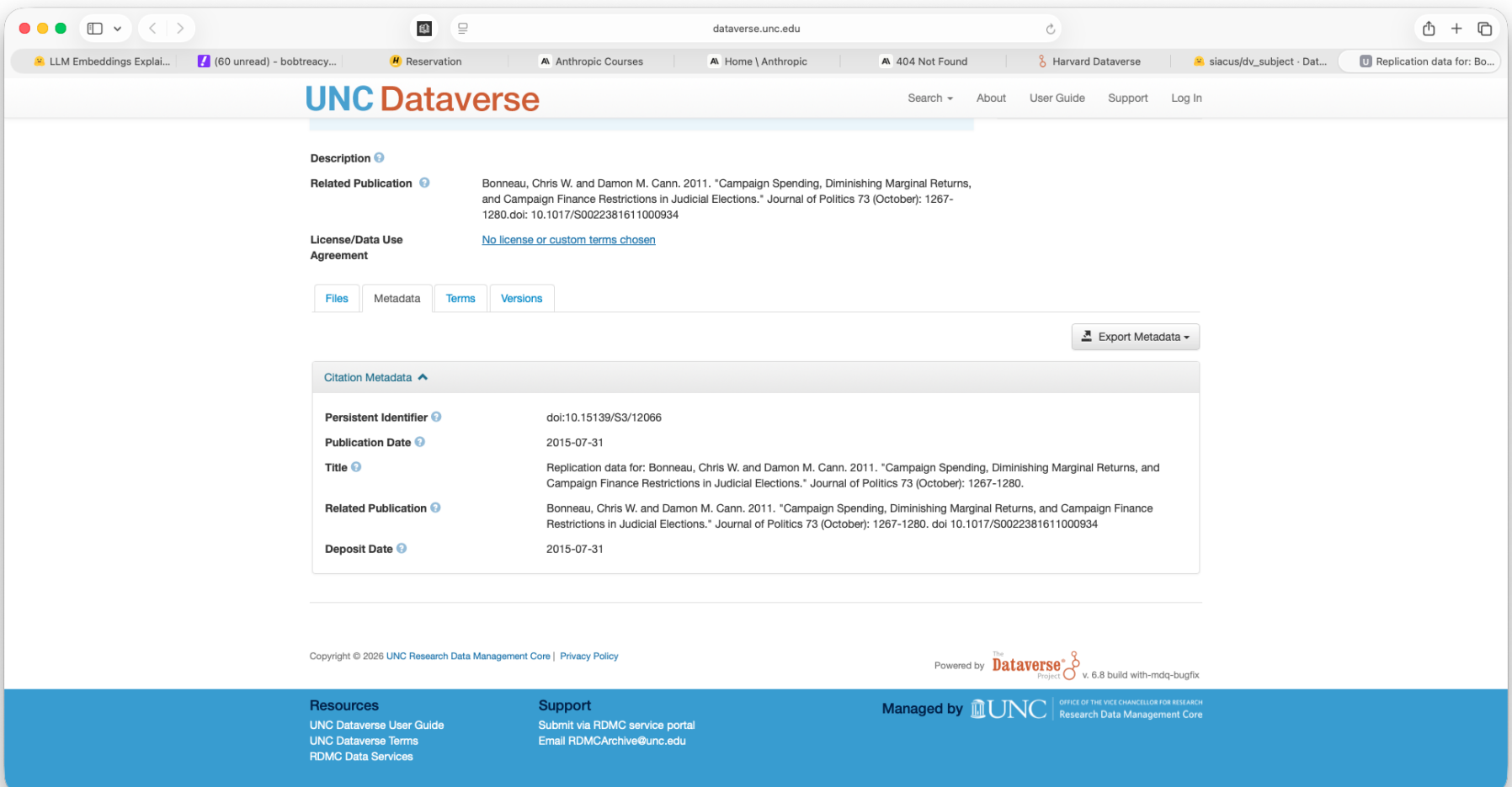
Harvard Dataverse



Dataset Metadata



Dataset Metadata



FAIR Data Principles

- Findability
- Accessibility
- Interoperability
- Reusability

Findability

Easy to find for humans and computers

Rich, machine-readable metadata

Unique identifiers

Accessibility



CLEAR PROTOCOLS
FOR ACCESSING DATA



AUTHENTICATION
AND AUTHORIZATION



OPEN METADATA

Interoperable

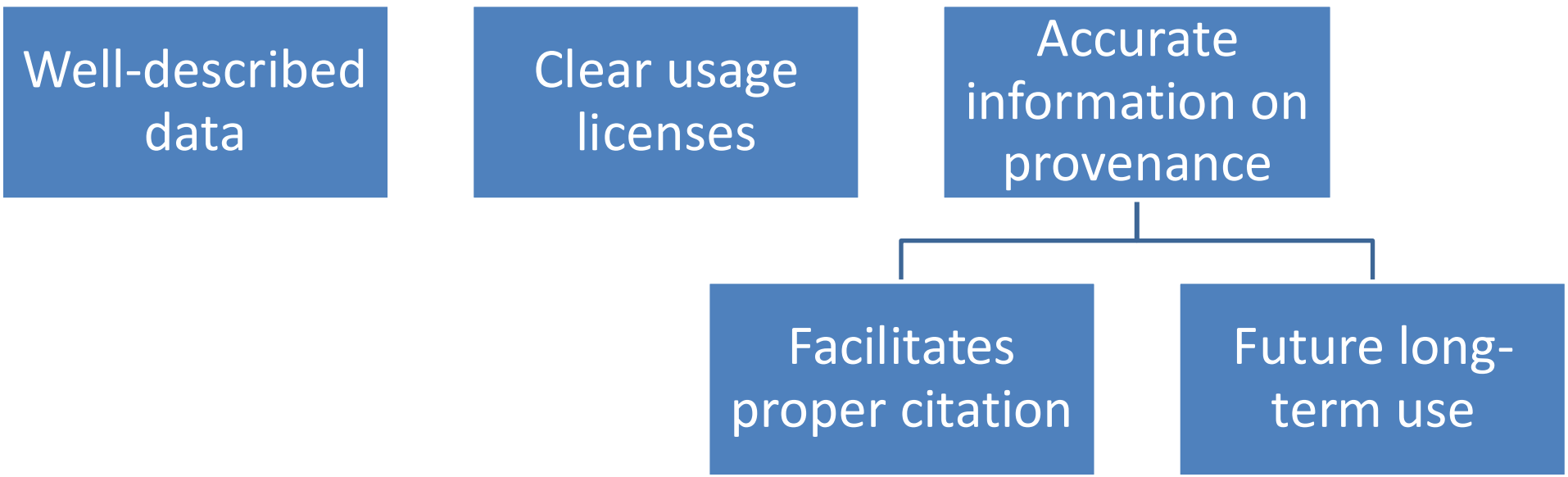


STANDARD STRUCTURED FORMATS



INTEGRATION WITH OTHER DATA, APPLICATIONS OR WORKFLOWS

Reusable


Well-described data
Clear usage licenses
Accurate information on provenance
Facilitates proper citation
Future long-term use

Retrieval Augmented Generation



EXTERNAL DATA GROUNDS RESPONSES



DOMAIN-SPECIFIC ANSWERS

Knowledge Graph

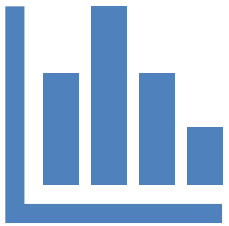
GraphRAG

- Smarter GenAI
- Semantic search

Neo4j



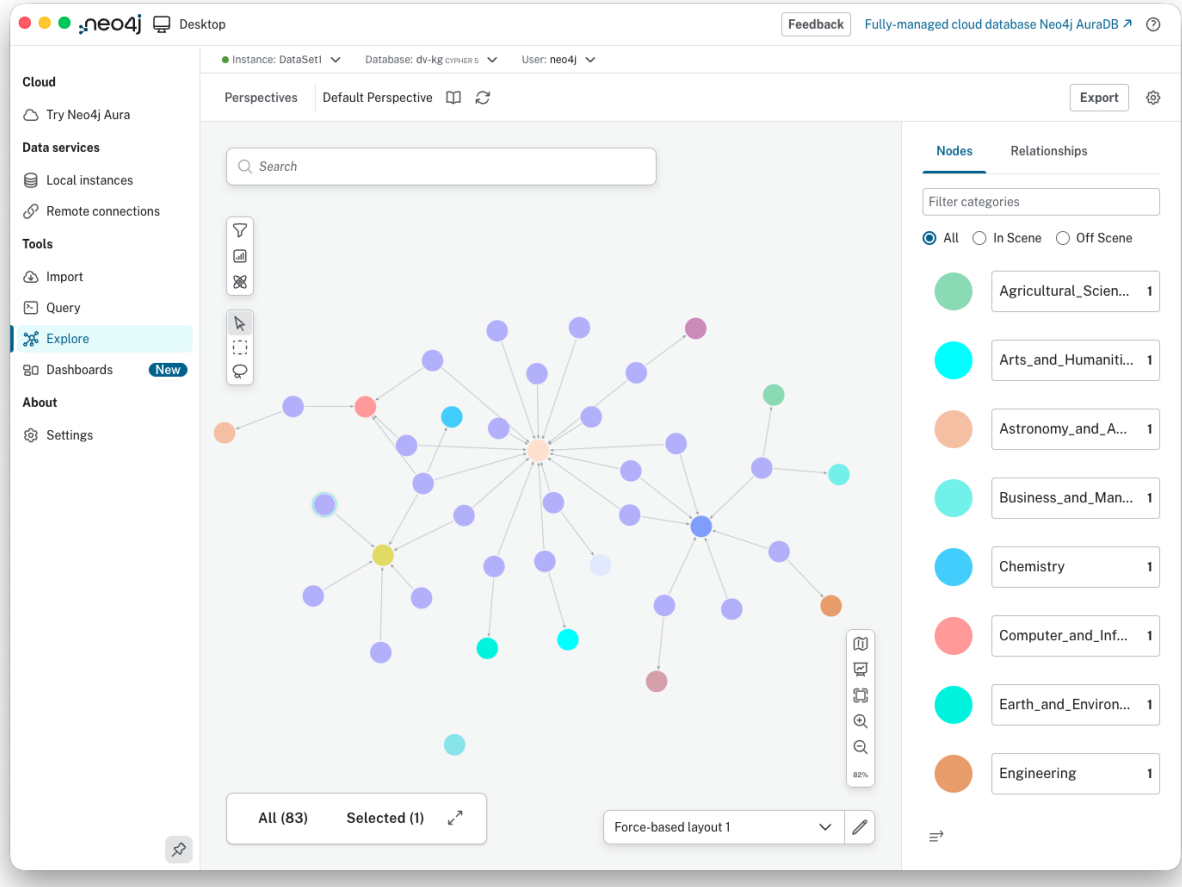
Graph database



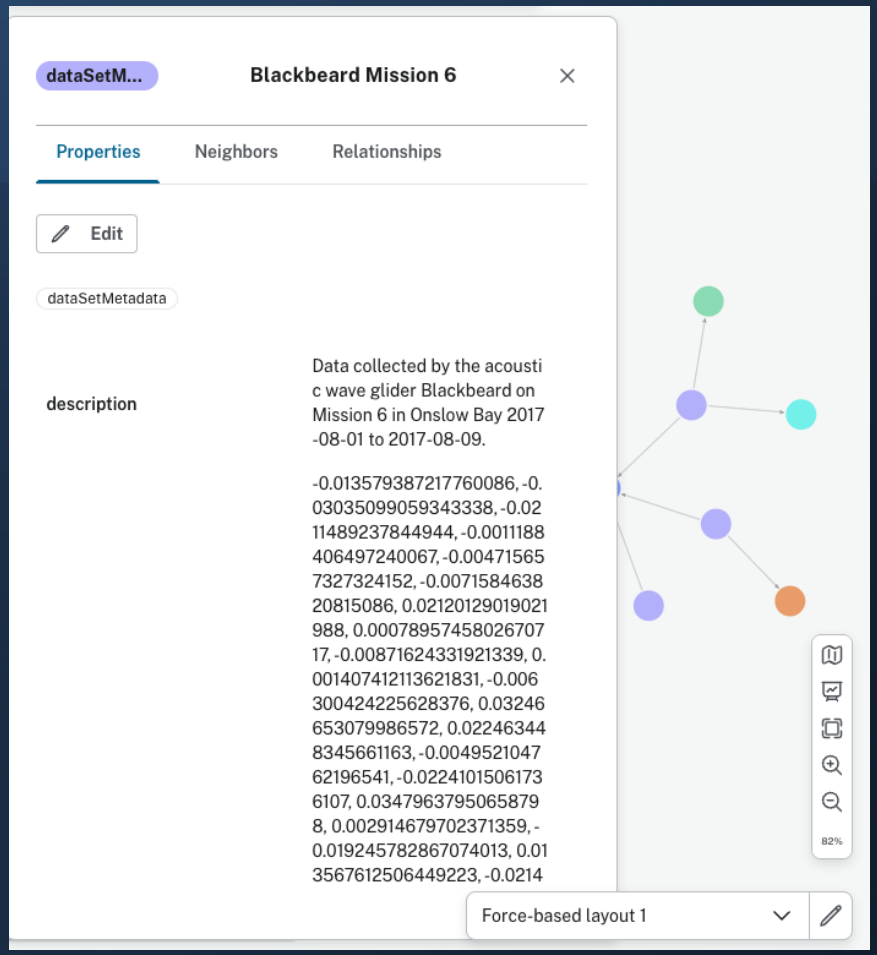
Cypher

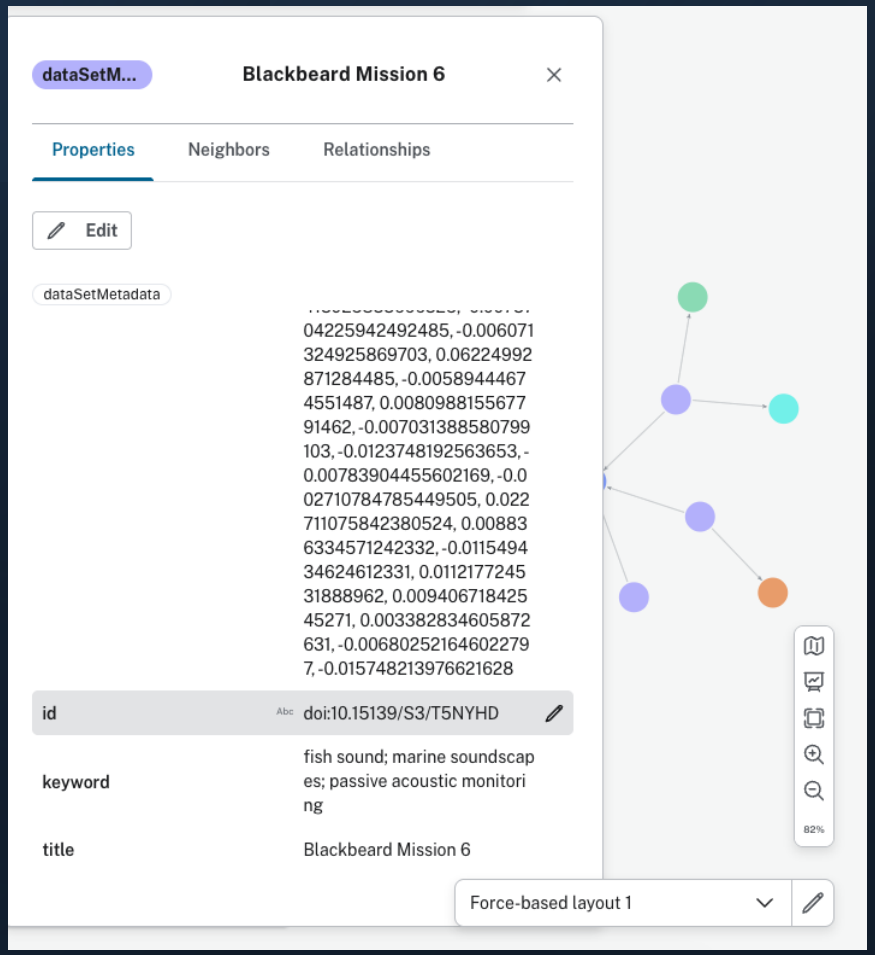
Declarative graph query language

Neo4j Desktop

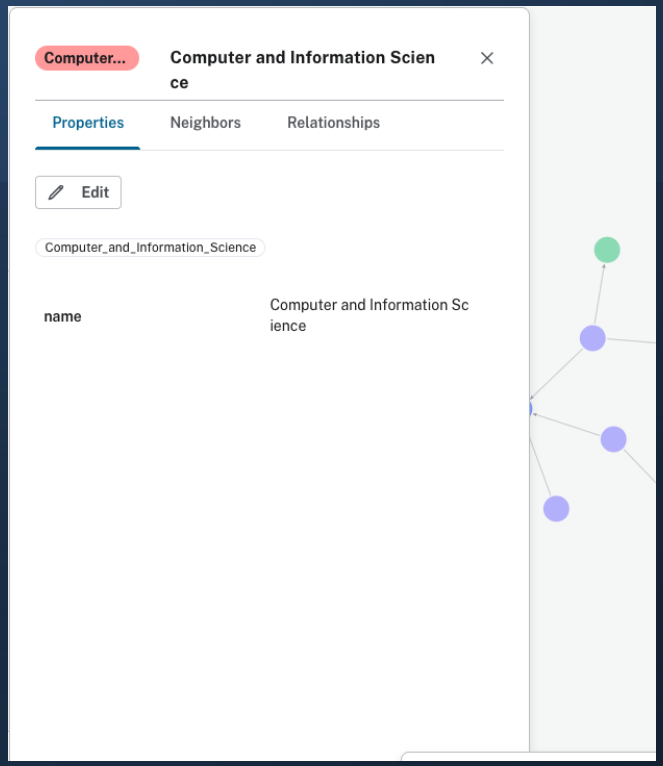


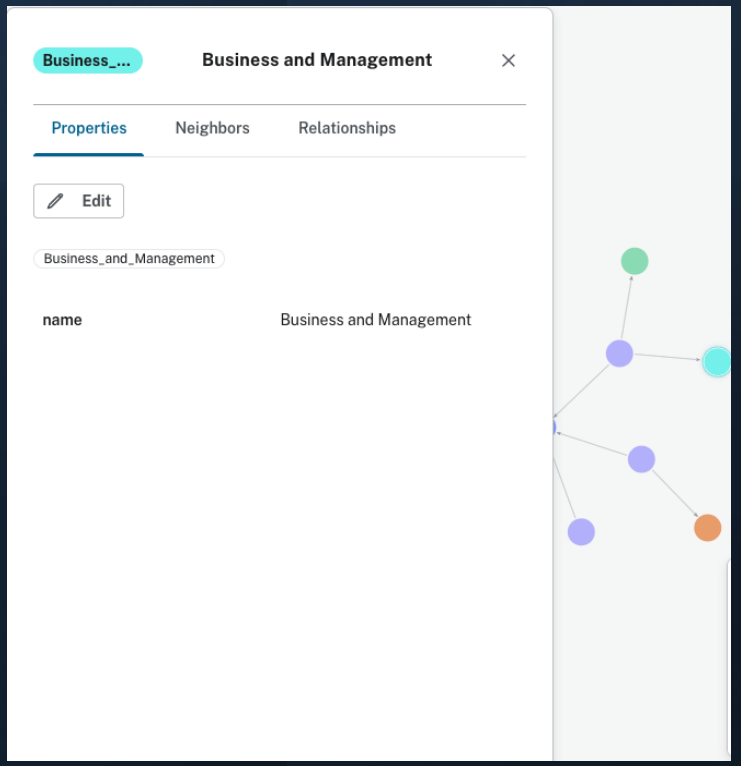
dataSetMetadata Node



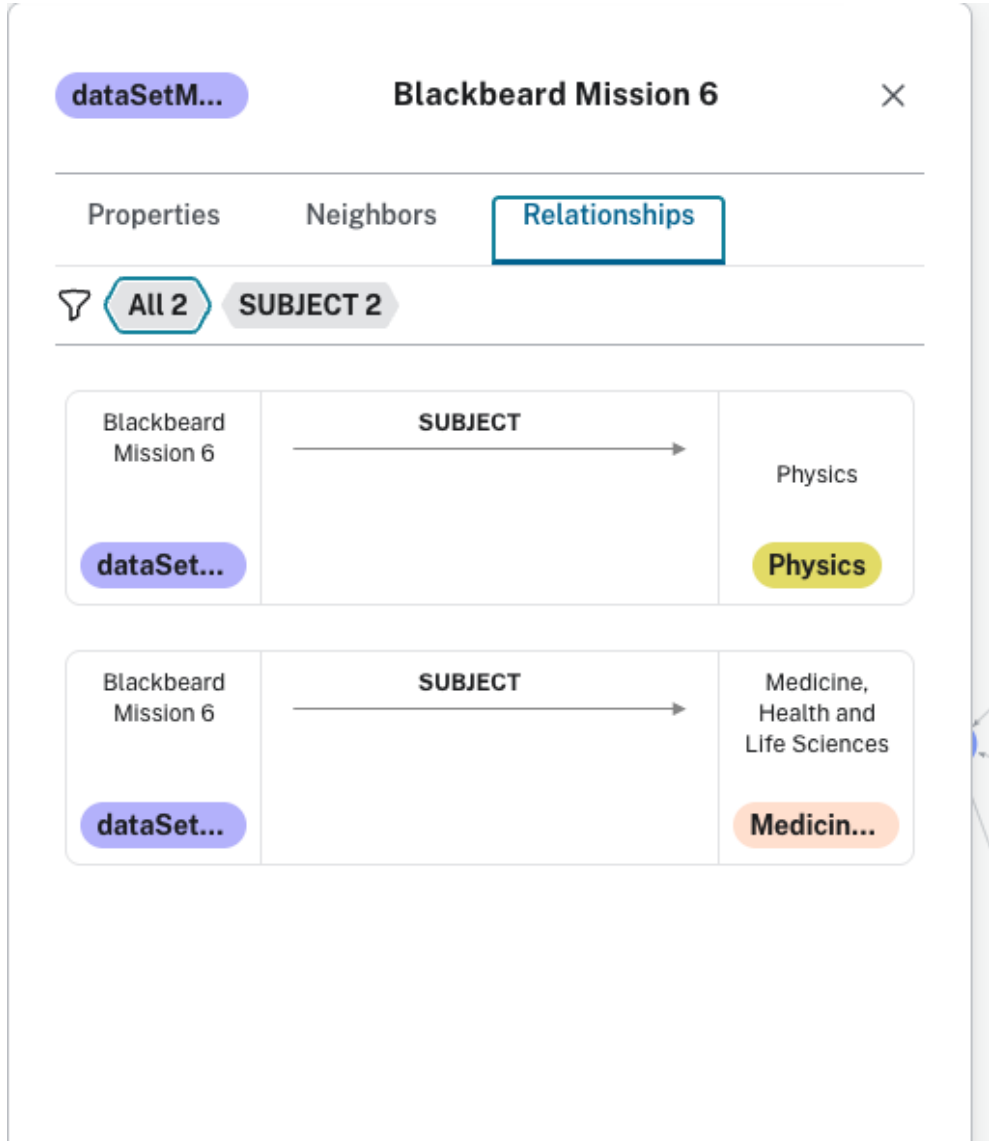


Subject nodes





Data Set Metadata Relationships



Create Nodes for each subject and relationships between metadata and subjects

String dvSubjects =
"Medicine;Law;Agricultural;Engineering;Business;Earth;Social;Astronomy;Arts;Mathematical;Physics;Chemistry;Computer;Other";

createSubjectRelationships(session, dvSubjects);

queryExtractCitationData(session, props);

Create Subject Nodes

```
private static void createSubjectRelationships(Session session, String dvSubjects) {
   // Ensure a single Root node exists
   session.run("MERGE (:Root {name:'Root'})");

… (some input validation) …
String[] parts = dvSubjects.split(";");
   int linked = 0;
   for (String raw : parts) {
      String label = raw.trim();
      … (some validation and transformation of raw for neo4j …
      String cypher = "MATCH (r:Root), (s:" + label + ") MERGE (r)-[:SUBJECT]->(s)";
      try {
         session.run(cypher);
         linked++;
      } catch (Exception e) { … }
   }

}
```

Get dataset metadata related to each subject and create nodes and relationships on neo4j

```
 private static void queryExtractCitationData(Session session, Properties props) throws Exception {
… (postgres setup stuff)
String dvSubjects =
"Medicine;Law;Agricultural;Engineering;Business;Earth;Social;Astronomy;Arts;Mathematical;Physics;Chem
istry;Computer;Other";
String queryString = "SELECT * FROM extractcitationdata WHERE \"Subject\" LIKE '%" + dvSubject + "%'
AND \"Keywords\" IS NOT NULL";

… ( get dataset metadatas related to each subject from postgres)
                String subjectField = rs.getString("Subject");
                 String description = rs.getString("Description");
                 String keywords = rs.getString("Keywords");
                 String id = rs.getString("doi");
                 String title = rs.getString("Title");


                 String[] subjects = subjectField.split(";");
                 String[] for (String s : subjects) {
                    upsertDataSetMetadataWithSubject(session, id, keywords, description, title, s);
                 }
   }
```

Upsert Dataset Metadata and link to subjects

```
 private static void upsertDataSetMetadataWithSubject(Session session, String id, String keyword, String description,
String title, String subject) {
… (validation and transformation for neo4j)
try {
        // Upsert the dataSetMetadata node
        session.run(new Query("MERGE (d:dataSetMetadata {id:$id}) SET d.keyword=$keyword,
d.description=$description, d.title=$title")
                .withParameters(java.util.Map.of(
                        "id", id,
                        "keyword", keyword,
                        "description", description,
                        "title", title
                )));

        // Ensure the subject-labeled node exists (also store its name for convenience)
        String mergeSubject = "MERGE (s:" + safeSubject + " {name:$name}) RETURN s";
        session.run(new Query(mergeSubject).withParameters(java.util.Map.of("name", subject.trim())));

        // Create the SUBJECT relationship
        safeSubject.replaceAll(",", " ").replaceAll(" ", "_");
        String relate = "MATCH (d:dataSetMetadata {id:$id}), (s:" + safeSubject + ") MERGE (d)-[:SUBJECT]->(s)";
        session.run(new Query(relate).withParameters(java.util.Map.of("id", id)));
    } catch (Exception e) { … }
  }
```

Embeddings

Embeddings have long existed in AI/Machine Learning and have had different form and function over time

- Traditional word embedding
  - TF-IDF
- Static Word Embedding
  - Word2Vec
- Contextualized Word Embedding (Transformer Models)
  - BERT
  - Llama
  - GPT

Embeddings

Traditional word embedding

- Term Frequency-Inverse Document Frequency
  - Karen Spärck Jones 1972
  - Term Frequency
    - How many times a term appears in a document
  - Inverse Document Frequency
    - How rare a term is across the document set

Embeddings

Static embedding

- Word2Vec (Google 2013)
  - Transforms words into numerical vectors to capture semantical relationships
  - Shallow two-layer networks

Embeddings

Contextualized Word Embedding

deeper networks capture word semantics in different contexts

river bank, money in the bank, bank a shot

- BERT (Google 2018)
  - Encoder only
- Llama
  - decoder
- GPT
  - decoder

Embeddings

The potential of medical image analysis with neural networks is limited by the restricted availability of extensive data sets. The incorporation of synthetic training data is one approach to bypass this shortcoming, as synthetic data offer accurate annotations and unlimited data size. We evaluated eleven CycleGAN for the synthesis of computed tomography (CT) images based on XCAT body phantoms.

-0.023407788947224617, 0.02044704556465149, -0.0521521121263504, 0.0011796851176768541, 0.005765027832239866, 0.0019748597405850887, 0.013511423021554947, 0.007508939132094383, -0.02533387579023838, -0.02055319771170616, 0.025186719372868538, 0.016050180420279503, 0.00430346978828311, 0.012786785140633583, -0.002833350794389844, 0.0263542328029871, 0.022029826417565346, 0.010322026908397675, -0.015952683985233307, -0.02927154116332531, 0.006223923061043024, -0.004685416352003813, -0.009926357306540012, 0.004619856830686331, -0.00850449688732624, 0.005997204687446356, -0.007136725354939699, -0.008976006880402565, -0.0019016742007806897, 0.019402306526899338, -0.03142813593149185, 0.021717462688684464, 0.026137441396713257, 0.02233792282640934, -0.01104722823947668, -0.020759668201208115, -0.005668578203767538, -0.009206852875649929, -0.010281765833497047, 0.004569460637867451, 0.0016325259348377585, 0.0008355994359590113, 0.023947348818182945, 0.027546895667910576, -0.016025658696889877, 0.01069233100861311, 0.007009674794971943, 0.012422394938766956, 0.0035674097016453743, 0.004643699619919062, 0.01123669184744358, …

Create embeddings in Neo4j

```
public int generateAndSaveEmbeddings(String nodeLabel, String propertyName, int limit, boolean overwrite) {
   // Get nodes that need embeddings
   List<EncodeRequest> nodesToEmbed = neo4jService.getNodesForEmbedding(nodeLabel, propertyName, limit, overwrite);
   List<Map<String, Object>> embeddingsToSave = new ArrayList<>();
   for (EncodeRequest request : nodesToEmbed) {
      try {
         float[] embedding = ollamaEmbeddingService.generateEmbedding(request.getText());
         List<Double> embeddingList = new ArrayList<>();
         for (float f : embedding) {
            embeddingList.add((double) f);
         }
         Map<String, Object> embeddingData = new HashMap<>();
         embeddingData.put("id", request.getId());
         embeddingData.put("embedding", embeddingList);
         embeddingsToSave.add(embeddingData);
      } catch (Exception e) {...}
   }
   if (!embeddingsToSave.isEmpty()) {
      neo4jService.saveNodeEmbeddings(embeddingsToSave);
   }
   return embeddingsToSave.size();
}
```

Get Nodes for embedding

```
public List<EncodeRequest> getNodesForEmbedding(String nodeLabel, String propertyName, int limit, boolean overwrite) {
    setup();

    // Validate inputs to prevent Cypher injection
    if (!nodeLabel.matches("[A-Za-z_][A-Za-z0-9_]*")) {
        throw new IllegalArgumentException("Invalid node label");
    }
    if (!propertyName.matches("[A-Za-z_][A-Za-z0-9_]*")) {
        throw new IllegalArgumentException("Invalid property name");
    }

    String whereEmbedding = overwrite ? "TRUE" : "n.embedding IS NULL";
    String cypher = String.format("""
            MATCH (n:%s)
            WHERE %s AND n.%s IS NOT NULL AND trim(n.%s) <> ''
            RETURN elementId(n) AS id, n.%s AS text
            LIMIT $limit
            """, nodeLabel, whereEmbedding, propertyName, propertyName, propertyName);

    SessionConfig config = SessionConfig.builder().withDatabase(configuration.getDatabase()).build();
    try (Session session = driver.session(config)) {
        return session.executeRead(tx -> {
            List<EncodeRequest> out = new ArrayList<>();
            var result = tx.run(cypher, Map.of("limit", limit <= 0 ? 1000 : limit));
            while (result.hasNext()) {
                var rec = result.next();
                String id = rec.get("id").asString();
                String text = rec.get("text").asString("");
                out.add(new EncodeRequest(text, id));
            }
            return out;
        });
    } catch (Exception e) {
        e.printStackTrace();
    }
    return List.of();
}
```

Save Node Embeddings

```
public void saveNodeEmbeddings(List<Map<String, Object>> embeddings) {
    setup();
    String cypher = """
        UNWIND $data AS row
        WITH row
        MATCH (n) WHERE elementId(n) = row.id
        CALL db.create.setNodeVectorProperty(n, 'embedding', row.embedding)
    """;
    SessionConfig config =
SessionConfig.builder().withDatabase(configuration.getDatabase()).build();
    try (Session session = driver.session(config)) {
        session.executeWriteWithoutResult(tx -> tx.run(cypher, Map.of("data", embeddings)));
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Similarity Metrics

- Cosine
- Euclidean

Find Similar Nodes

```
/**
 * Find similar nodes based on a text query using cosine similarity.
 *
 * @param nodeLabel The label of nodes to search
 * @param queryText The text to find similar nodes for
 * @param topK Number of similar nodes to return
 * @return List of similar nodes with their properties and similarity scores
 */
public List<Map<String, Object>> findSimilarNodesByText(String nodeLabel, String queryText, int
topK) {
    // Generate embedding for the query text
    float[] queryEmbedding = ollamaEmbeddingService.generateEmbedding(queryText);

    // Find similar nodes using cosine similarity
    return neo4jService.findSimilarNodesCosine(nodeLabel, queryEmbedding, topK);
}
```

Embed input description

```
public float[] generateEmbedding(String text) {
    EmbeddingModel embeddingModel = embeddingModelProvider.getIfAvailable();
    if (embeddingModel == null) {
        throw new IllegalStateException("… Ensure an embedding model (e.g., Ollama) is configured.");
    }
    return embeddingModel.embed(text);
}
```

Cosine Similarity (1)

```
public List<Map<String, Object>> findSimilarNodesCosine(String nodeLabel, float[] embedding, int topK) {
    setup();


    if (!nodeLabel.matches("[A-Za-z_][A-Za-z0-9_]*")) {
        throw new IllegalArgumentException("Invalid node label");
    }

    // Convert float[] to List<Double> for Neo4j parameter
    List<Double> embeddingList = new ArrayList<>();
    for (float f : embedding) {
        embeddingList.add((double) f);
    }

    String cypher = String.format("""
            MATCH (n:%s)
            WHERE n.embedding IS NOT NULL
            WITH n, vector.similarity.cosine(n.embedding, $embedding) AS score
            ORDER BY score DESC
            LIMIT $topK
            WITH elementId(n) AS id, properties(n) AS props, score
            RETURN id, apoc.map.removeKey(props, 'embedding') AS properties, score
            """, nodeLabel);
```

Cosine Similarity (2)

```
SessionConfig config = SessionConfig.builder().withDatabase(configuration.getDatabase()).build();
  try (Session session = driver.session(config)) {
    return session.executeRead(tx -> {
      List<Map<String, Object>> results = new ArrayList<>();
      var result = tx.run(cypher, Map.of("embedding", embeddingList, "topK", topK));
      while (result.hasNext()) {
        var rec = result.next();
        Map<String, Object> item = new HashMap<>();
        item.put("id", rec.get("id").asString());
        item.put("properties", rec.get("properties").asMap());
        item.put("score", rec.get("score").asDouble());
        results.add(item);
      }
      return results;
    });
  } catch (Exception e) {
    e.printStackTrace();
  }
  return List.of();
}
```

Get the set of subjects

```
public Set<String> aggregateNamesFromSimilarNodesResults(List<Map<String,
Object>> similarNodesResults) {
    if (similarNodesResults == null || similarNodesResults.isEmpty()) {
        return new HashSet<>();
    }

    List<String> nodeIds = new ArrayList<>();
    for (Map<String, Object> result : similarNodesResults) {
        Object id = result.get("id");
        if (id != null) {
            nodeIds.add(id.toString());
        }
    }

    return aggregateNamesFromSubjectRelations(nodeIds);
}
```

Get the set of subjects(2)

```
public Set<String> aggregateNamesFromSubjectRelations(List<String> nodeIds) {
    setup();
    if (nodeIds == null || nodeIds.isEmpty()) {
        return new HashSet<>();
    }

    String cypher = """
            UNWIND $nodeIds AS nodeId
            MATCH (n)-[r]->(m)
            WHERE elementId(n) = nodeId AND m.name IS NOT NULL
            RETURN DISTINCT m.name AS name
            """;

    SessionConfig config = SessionConfig.builder().withDatabase(configuration.getDatabase()).build();
    try (Session session = driver.session(config)) {
        return session.executeRead(tx -> {
            Set<String> names = new HashSet<>();
            var result = tx.run(cypher, Map.of("nodeIds", nodeIds));
            while (result.hasNext()) {
                var rec = result.next();
                String name = rec.get("name").asString("");
                if (!name.isEmpty()) {
                    names.add(name);
                }
            }
            return names;
        });
    } catch (Exception e) {...}
    return new HashSet<>();
}
```

Demo

References

Building Neo4j-Powered Applications with LLMs
Ravindranatha Anthapu, Siddhant Agarawal
Packt Publishing 2025

https://neo4j.com/docs/

https://www.go-fair.org/wp-content/uploads/2022/01/FAIRPrinciples_overview.pdf

https://dataverse.org/about

https://github.com/rtreacy/DataverseKGRAGSpringAI

Q & A